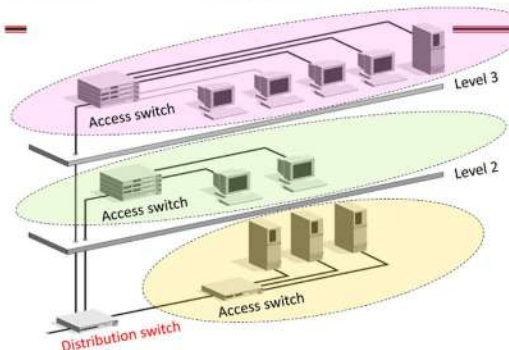


Lab 1 Notes

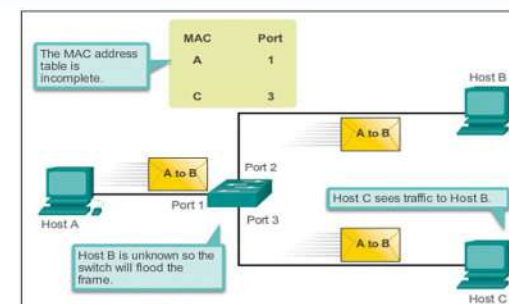
In real world networks, the first entry point for (insider) attackers are usually the LAN switches, or access switches.



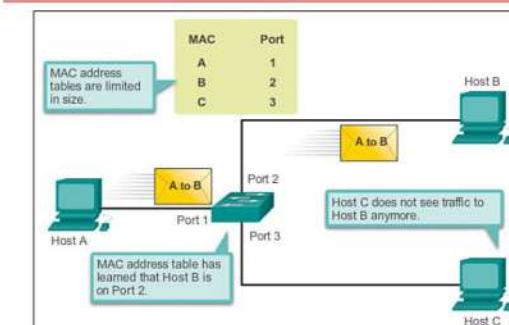
Hence, it's important to secure LAN switches because compromising layer 2 will compromise layer 3 and above which are encapsulated within it.



Recall from ICT1010 about the switch's MAC address table. If the destination MAC address of a frame is not found in the table, it will be flooded out to all ports except incoming port.



After address self-learning, a unicast frame will only be forwarded out to the intended port, and not others. Thus, a switch is considered to be more 'secure' than a hub.



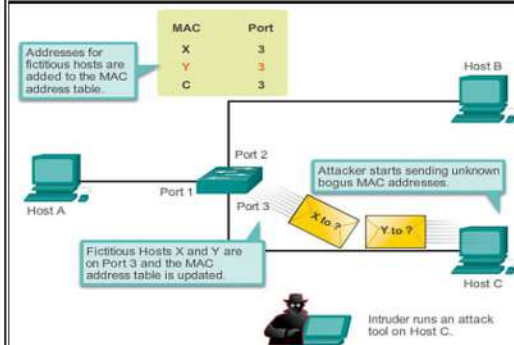
However, the switch's MAC address table cannot be of infinite size. What happens when a switch does not have sufficient memory to store more new MAC addresses?

E.g.: The address table size of various models of Cisco switches

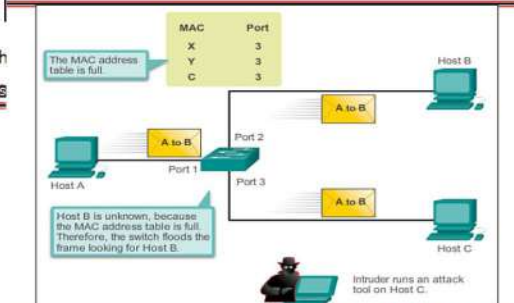
Switch Model	Number of Bridge-Table Entries
Cisco Catalyst Express 500	8000
Cisco Catalyst 2948G	16,000
Cisco Catalyst 2940/50/55/60/70	Up to 8000
Cisco Catalyst 3500XL	8192
Cisco Catalyst 3550/60	Up to 12,000 (depending on the model)
Cisco Catalyst 3750/3750M	12,000
Cisco Catalyst 4500	32,768
Cisco Catalyst 4948	55,000
Cisco Catalyst 6500/7600	Up to 131,072 (more if distributed feature cards are installed)

Note: The actual address table size of the different switches in the lab can be different. Find out in lab.

MAC flooding attack: bombarding the switch with frames containing fake source MAC addresses until the MAC address table is full.



As a result, the destination address of an authentic frame is likely not to be found in the address table, and it will be flooded and received by attackers!



A common tool for conducting MAC flooding attack is macof (in Kali Linux) which sends out frames with randomly generated source and destination MAC addresses.

Refer to <http://linux.die.net/man/8/macof> on the use of macof:

```
[root@client root]# macof -n 5
3a:50:0d:31:f9:02 75:83:21:6a:ca:f1 0.0.0.0.30571 > 0.0.0.0.19886: S
212769628:212769628(0) win 512
db:ad:aa:20:ac:19 f6:fe:a7:25:4b:9a 0.0.0.0.4842 > 0.0.0.0.13175: S
1394722674:1394722674(0) win 512
2b:0e:b1:46:a0:50 d9:9e:b1:1f:8f:8f 0.0.0.0.32533 > 0.0.0.0.29366: S
129383321:129383321(0) win 512
ce:56:ee:19:85:1a 39:56:a8:38:52:de 0.0.0.0.26508 > 0.0.0.0.8634: S
86470327:86470327(0) win 512
89:63:da:11:97 55:8b:ef:5d:34:92 0.0.0.0.54679 > 0.0.0.0.44152: S
1851212987:1851212987(0) win 512
[root@client root]#
```

Far-Left/High-Order-Octet Value in Binary	
2B	0010 1011
DB	1101 1011
89	1000 1001

Here's an example of the switch's address table before and after macof attack in previous slide.

MAC address table before attack:

```
6K-1-720# sh mac address-table dynamic vlan 20
Legend: * - primary entry
age - seconds since last seen
n/a - not available

vlan mac address type learn age ports
* 20 00ff.01ff.01ff dynamic Yes 45 Gi1/15
```

MAC address table after attack:

```
6K-1-720# sh mac address-table dynamic vlan 20
Legend: * - primary entry
age - seconds since last seen
n/a - not available

vlan mac address type learn age ports
* 20 ce56.ee19.851a dynamic Yes 70 Gi1/15
* 20 00ff.01ff.01ff dynamic Yes 70 Gi1/15
* 20 3a50.db3f.e9c2 dynamic Yes 70 Gi1/15
```

Why only 2 new MAC addresses out of 5 are learnt?

Here's an example of the switch's address table before and after macof attack in previous slide.

MAC address table before attack:

```
6K-1-720# sh mac address-table dynamic vlan 20
Legend: * - primary entry
age - seconds since last seen
n/a - not available

vlan mac address type learn age ports
* 20 00ff.01ff.01ff dynamic Yes 45 Gi1/15
```

MAC address table after attack:

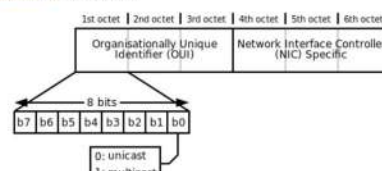
```
6K-1-720# sh mac address-table dynamic vlan 20
Legend: * - primary entry
age - seconds since last seen
n/a - not available

vlan mac address type learn age ports
* 20 ce56.ee19.851a dynamic Yes 70 Gi1/15
* 20 00ff.01ff.01ff dynamic Yes 70 Gi1/15
* 20 3a50.db3f.e9c2 dynamic Yes 70 Gi1/15
```

Why only 2 new MAC addresses out of 5 are learnt?

Note that multicast addresses will not be learned by the switch. (Recall from ICT1010 Lec 2 on how to identify multicast frames as shown below.)

In 802.3 standard, bit 0 (b0) of the first octet is set to 1 to indicate a multicast MAC address:



Note also the entries in the MAC address table will be removed automatically if not seen again within the aging time, default to 300 seconds.

To configure aging time for entries in MAC address table:

```
switch(config)# mac address-table aging-time seconds [ vlan vlan-id ]
```

seconds : range from 10 to 1000000 seconds

To return to default setting:

```
switch(config)# no mac address-table aging-time
```

To disable aging time:

```
switch(config)# mac address-table aging-time 0 [ vlan vlan-id ]
```

So to be successful in MAC flooding attack, attackers will need to flood sufficient frames within the aging time.

Count of MAC address table entries before attack:

```
6K-1-720# clear mac address-table dynamic
MAC entries cleared.

6K-1-720# show mac address-table count
MAC Entries for all vlans : 37
Dynamic Address Count: 37
Static Address (User-defined) Count: 494
Total MAC Addresses In Use: 531
Total MAC Addresses Available: 65536
```

Count of MAC address table entries after attack:

```
6K-1-720# show mac address-table count
MAC Entries for all vlans : 58224
Dynamic Address Count: 58224
Static Address (User-defined) Count: 503
Total MAC Addresses In Use: 58727
Total MAC Addresses Available: 65536
```

So to be successful in MAC flooding attack, attackers will need to flood sufficient frames within the aging time.

Count of MAC address table entries before attack:

```
6K-1-720# clear mac address-table dynamic
MAC entries cleared.

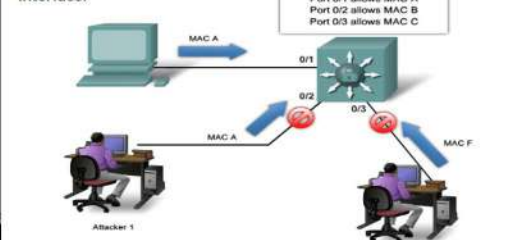
6K-1-720# show mac address-table count
MAC Entries for all vlans : 37
Dynamic Address Count: 37
Static Address (User-defined) Count: 494
Total MAC Addresses In Use: 531
Total MAC Addresses Available: 65536
```

Count of MAC address table entries after attack:

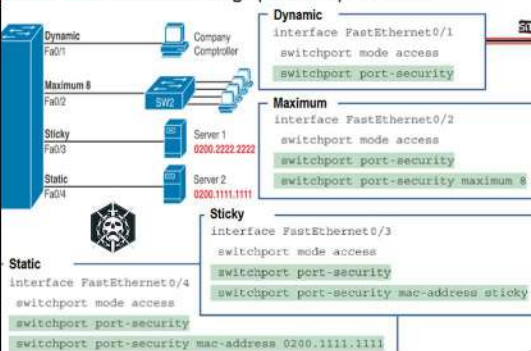
```
6K-1-720# show mac address-table count
MAC Entries for all vlans : 58224
Dynamic Address Count: 58224
Static Address (User-defined) Count: 503
Total MAC Addresses In Use: 58727
Total MAC Addresses Available: 65536
```

To prevent MAC flooding attack: enable Port Security in switches.

Port security restricts input to a switch's interface by identifying and limiting the MAC addresses of the hosts that are allowed to access the interface.



In Cisco switches, port security may be configured with one or a combination of the following options. Try out in lab.



Explanation of the port-security commands:

Command	Mode/Purpose/Description
switchport port-security mac-address mac-address	Interface configuration mode command that statically adds a specific MAC address as an allowed MAC address on the interface.
switchport port-security mac-address sticky	Interface subcommand that tells the switch to learn MAC addresses on the interface and add them to the configuration for the interface as secure MAC addresses.
switchport port-security maximum value	Interface subcommand that sets the maximum number of static secure MAC addresses that can be assigned to a single interface.

verify port-security interface fastEthernet 0/3 or show port-security interface commands as follows:

```
SW1# show port-security interface fastEthernet 0/3
Port Security          : Enabled
Port Status            : Secure-up
Violation Mode         : Shutdown
Aging Time             : 0 mins
Aging Type             : Absolute
SecureStatic Address Aging : Disabled
Maximum MAC Addresses  : 1
Total MAC Addresses    : 1
Configured MAC Addresses : 1
Sticky MAC Addresses   : 1
Last Source Address:Vlan : 0200.2222.2222:1
Security Violation Count : 0
```

In addition, there are 3 modes to configure the actions to take if port security violation is detected.

Command	Mode/Purpose/Description
switchport port-security violation [protect restrict shutdown]	Interface subcommand that tells the switch what to do if an inappropriate MAC address tries to access the network through a secure switch port.

Option on the switchport port-security violation Command	Protect	Restrict	Shutdown*
Discards offending traffic	Yes	Yes	Yes
Sends log and SNMP messages	No	Yes	Yes
Disables the interface, discarding all traffic	No	No	Yes

*Shutdown is the default mode

For port security violation shutdown mode, the port will be moved to error disabled state if violation is detected.

Line Status	Protocol Status	Interface Status	Typical Root Cause
Down	Down (err-disabled)	err-disabled	Port security has disabled the interface.

Two ways to recover err-disabled ports:

1. Manual recovery (more control)

```
Switch(config)# interface type num/num
Switch(config-if)# shutdown
Switch(config-if)# no shutdown
```

2. Auto recovery (to reduce administrative overhead)

```
Switch(config)# errdisable recovery cause psecure-violation
Switch(config)# errdisable recovery interval 600
```

Lab Exercise 2

2.1 Switch spoofing attack

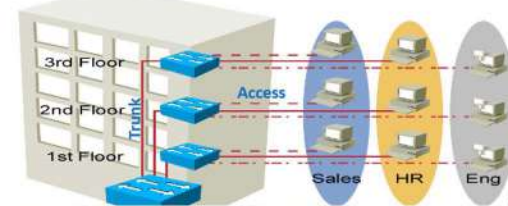
2.2 Defenses:

- Manually configure ports as access or trunk
- Disable DTP
- Good practice to shut down all unused ports and configure them into an unused VLAN

In ICT1010, you have learned about configuring VLANs (virtual LANs) on switches to separate frames in different VLANs to support users in different departments.

With VLANs, switch port connected to a link is operating either as:

- Access – carry frames for only one VLAN configured on the port; or
- Trunk – carry frames for multiple VLANs on one physical link



To simplify configuration, modern switches are commonly shipped with mechanisms or protocols to automate trunk formation; e.g. the Cisco Dynamic Trunking Protocol (DTP).

In Cisco switches, each switch port can be configured in 4 different modes: access, dynamic auto, trunk and dynamic desirable.

Then, DTP will be run automatically which will result in the link being an access or trunk as follows:

Administrative Mode	Access	Dynamic Auto	Trunk	Dynamic Desirable
access	Access	Access	Do Not Use ¹	Access
dynamic auto	Access	Access	Trunk	Trunk
trunk	Do Not Use ¹	Trunk	Trunk	Trunk
dynamic desirable	Access	Trunk	Trunk	Trunk

By default, Cisco switch ports are in dynamic auto mode which enables DTP and will passively wait to receive trunk negotiation to become a trunk.

To simplify configuration, modern switches are commonly shipped with mechanisms or protocols to automate trunk formation; e.g. the Cisco Dynamic Trunking Protocol (DTP).

In Cisco switches, each switch port can be configured in 4 different modes: access, dynamic auto, trunk and dynamic desirable.

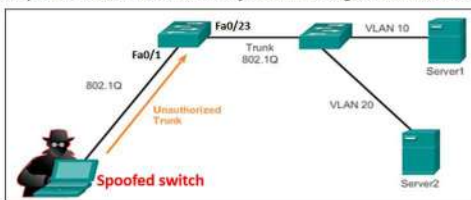
Then, DTP will be run automatically which will result in the link being an access or trunk as follows:

Administrative Mode	Access	Dynamic Auto	Trunk	Dynamic Desirable
access	Access	Access	Do Not Use ¹	Access
dynamic auto	Access	Access	Trunk	Trunk
trunk	Do Not Use ¹	Trunk	Trunk	Trunk
dynamic desirable	Access	Trunk	Trunk	Trunk

By default, Cisco switch ports are in dynamic auto mode which enables DTP and will passively wait to receive trunk negotiation to become a trunk.

Unfortunately, the use of DTP can lead to switch spoofing attack – a form of VLAN hopping attack where attackers could access frames in other VLANs that normally would not be accessible.

Taking advantage of the default configuration of the switch port which is dynamic auto, an attacker can inject DTP messages to form a trunk.



With the spoofed switch, the attacker may now receive frames from other VLANs, or send spoofed frames tagged for any target VLAN. An example of the result of conducting switch spoofing attack on a switch using Yersinia.

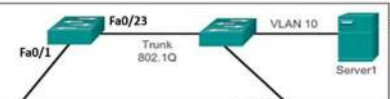
```
COM19600@out - Test VM
File Edit Setup Control Window Help
Switch#show interface fa0/1 trunk
Port      Mode      Negotiation  Status  Native vlan
Fa0/1    auto      802.1q      not-trunking  1
Port      Vlans allowed on trunk
Fa0/1    1-4094
Port      Vlans allowed and active in management domain
Fa0/1    1,10-20
Port      Vlans in spanning tree forwarding state and not pruned
Fa0/1    1,10-20
Switch#
```

Port Fa0/1 in default dynamic auto before attack:

```
COM19600@out - Test VM
File Edit Setup Control Window Help
Switch#show interface trunk
Port      Mode      Negotiation  Status  Native vlan
Fa0/1    auto      802.1q      not-trunking  1
Port      Vlans allowed on trunk
Fa0/1    1-4094
Port      Vlans allowed and active in management domain
Fa0/1    1,10-20
Port      Vlans in spanning tree forwarding state and not pruned
Fa0/1    1,10-20
Switch#
```

Port Fa0/1 becomes trunk after attack:

To prevent switch spoofing attack, disable DTP negotiations and manually configure ports as access or trunk explicitly.



```
SW2(config)# interface fa0/1
! Specifies that this is an access port
SW2(config-if)# switchport mode access
SW2(config-if)# switchport access vlan 10
! Note, even for an interface configured as an access port,
! negotiation packets are still being sent unless we disable it
SW2(config-if)# switchport nonegotiate
```

```
SW2(config-if)# interface fa0/23
! Specifies the port as a trunk
SW2(config-if)# switchport mode trunk
SW2(config-if)# switchport nonegotiate
```

For added security, it is also wise to shutdown all unused ports and place them in an unused VLAN, e.g. 100, so as to prevent someone from discovering a live port that might be exploited.

For example, if port Fa0/2 is unused, then shut it down to prevent attacker from discovering and exploiting it.



```
SW2(config)# interface fa0/2
SW2(config-if)# switchport access vlan 100
SW2(config-if)# shutdown

SW2(config-if)# interface fa0/23
SW2(config-if)# switchport trunk allowed vlan remove 100
```

Lab Exercise 3

3.1 Double tagging attack

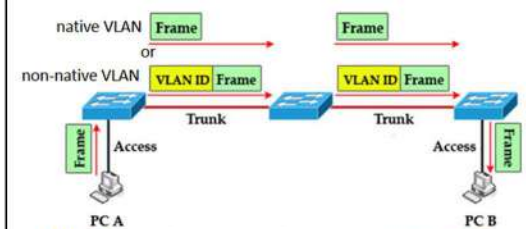
3.2 Defenses:

- Configure dedicated native VLAN for trunk which must not be used for access ports
- Enable tagging for native VLAN if applicable
- Good practice not to use default VLAN 1

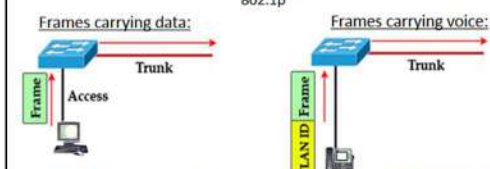
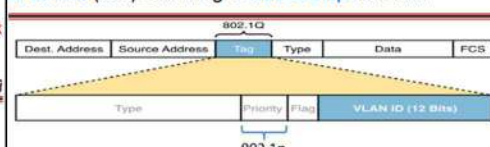
Inside a trunk, the different VLAN traffic are distinguished using the IEEE 802.1Q tagging standard. (Refer ICT1010 lecture 3 for details.)

Specifically, IEEE 802.1Q supports two types of VLANs inside trunk:

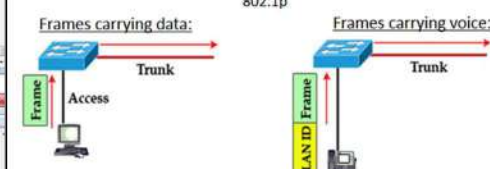
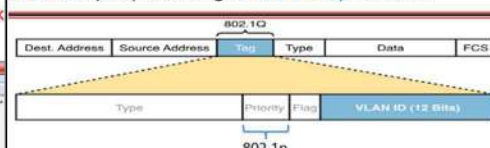
- one native VLAN – frames are not tagged
- all other VLANs – frames are tagged with appropriate VLAN ID



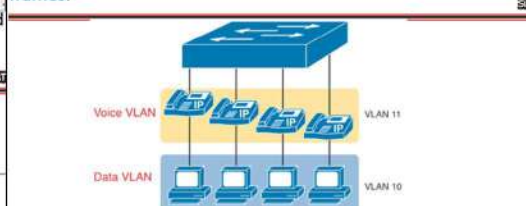
The 802.1Q tag is also being used to support voice over IP (VoIP), specifically the 3-bit priority field is used to specify class of service (CoS) according to IEEE 802.1p standard.



The 802.1Q tag is also being used to support voice over IP (VoIP), specifically the 3-bit priority field is used to specify class of service (CoS) according to IEEE 802.1p standard.

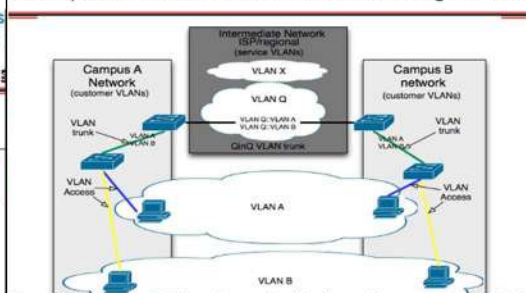


To support VoIP, the switch ports will also need to be configured with a data VLAN and an auxiliary voice VLAN to receive tagged frames.

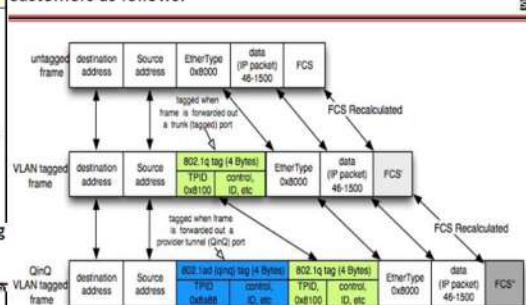


```
SW1(config-vlan)# interface range FastEthernet0/1 - 4
SW1(config-if)# switchport mode access
SW1(config-if)# switchport access vlan 10
SW1(config-if)# switchport voice vlan 11
```

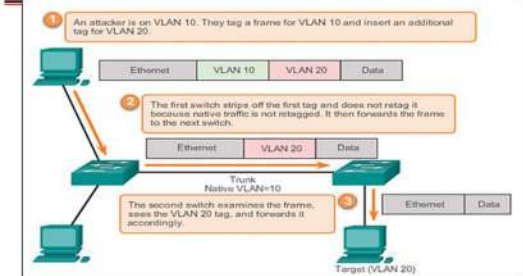
With the development of metropolitan-area Ethernet, double 802.1Q tags are also supported such as Q-in-Q (802.1Q in 802.1Q) where customers' VLANs can tunnel through ISP VLANs.



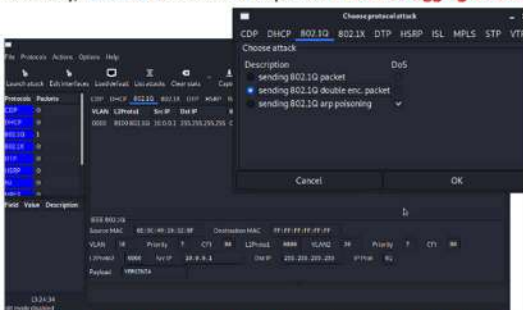
Specifically, Q-in-Q is implemented by inserting an outer 802.1Q tag of the ISP besides the original inner 802.1Q tag of the customers as follows:



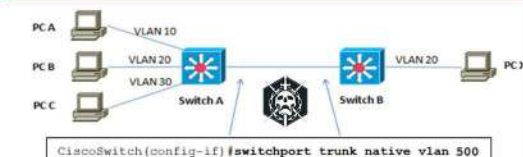
Double tagging attack – another form of VLAN hopping attack where an attacker can send frames to hop to other VLANs without going through a router.



Note: double tagging attack works only if attacker's access port has the same VLAN ID as the native VLAN of the trunk, default is VLAN 1. Similarly, Yersinia can be used to perform double tagging attack.



To prevent double tagging attack, configure dedicated native VLAN, e.g. VLAN 500, for all trunk ports which should not be assigned to any access ports.



Alternatively, some switches like Catalyst 3650 switch in the lab have the feature of forcing all traffic on the trunk to be tagged, including native VLAN.

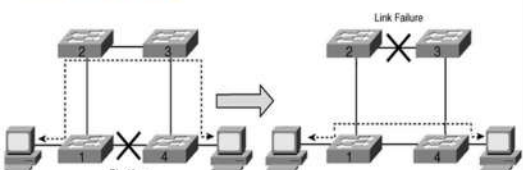
Lab Exercise 4

4.1 STP Attack

4.2 Defense:

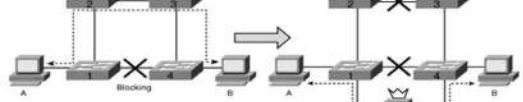
Implement PortFast, BPDU guard and root guard. Recall from ICT1010/ICT1013 to mitigate single point of failure, switches are commonly interconnected with redundancies and STP (802.1D) are run automatically to prevent loops.

With redundancies, the network can continue to work if a link fails as Spanning Tree Protocol (STP) is able to re-configure itself automatically.



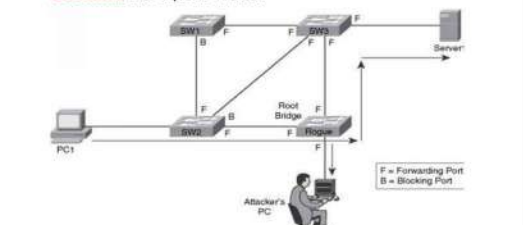
However, STP has no security. An attacker can inject BPDU with lower bridge ID to take over root switch and capture frames that otherwise would not be accessible.

Connecting an attacker host with 2 Ethernet ports, one to each switch as shown:



Before attack, frames between hosts A and B passes via switches 1-2-3-4 and are not seen by attacker. After STP attack, link 2-3 becomes blocked. Frames between A and B are now passing via the attacker's host (new root bridge) and are captured by attacker.

So for you to try STP attack in the lab, we will assume that an attacker is bringing his own rogue switch to take over original root switch and capture frames.



Note: for the rogue switch to be elected as root, you will need to configure the 4-bit priority as shown:

The configuration of priority value must be in multiples of 4096 from 0 to 61,440. (The first 4-bit of the value will be used to set the priority.)

SW2# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
SW2(config)# spanning-tree vlan 10 priority 24576
SW2(config)# *

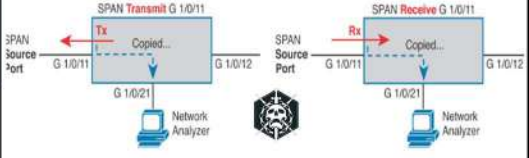
Alternatively, Cisco provides the root primary command which will set the priority low enough to become the root:

SW2# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
SW2(config)# spanning-tree vlan 10 root primary

- If current root priority > 24,576, switch priority is set to 24,576
- If current root priority ≤ 24,576, switch priority is set to the highest multiple of 4096 just below current root priority

In addition, to copy frames from the rogue switch to your laptop, you will need to configure Switch Port Analyzer (SPAN) as follows:

SPAN may be configured to capture only transmitted frames, or only received frames, or both, from a source port.



SW11(config)# monitor session 1 source interface Gi1/0/11 both
SW11(config)# monitor session 1 destination interface Gi1/0/21

The monitor session number may be any valid number from 1 to 18. If your capture does not appear to be working, you may verify the SPAN session with the following commands:



To prevent STP attack, apply BPDU guard feature available in Cisco switches.

BPDU guard may be enabled individually at each interface that is not supposed to receive BPDU, e.g.:

S2 (config)# interface Fa0/11
S2 (config-if)# spanning-tree bpduguard enable

Alternatively, BPDU guard may simply be enabled globally on all PortFast-enabled ports since these ports should not receive BPDU.

S2 (config)# spanning-tree portfast bpduguard default

If a BPDU guard-enabled port receives a BPDU, it will be put into error-disabled state. (The disabled port must then be re-enabled manually or configured with auto-recovery similar as port-security violation.)

Recall ICT1010/ICT1013 that a port will transit from listening to learning before finally in forwarding or blocked state to prevent loops.

State	Forwards Data Frames?	Learns MACs Based on Received Frames?	Transitory or Stable State?
Disabled	No	No	Stable
Blocking	No	No	Stable
Listening	No	No	Transitory
Learning	No	Yes	Transitory
Forwarding	Yes	Yes	Stable

For ports connected to user hosts which will not create loops, you may speed up transitioning to forwarding state immediately by configuring PortFast to bypass listening and learning states:

SW3 (config)# interface fastEthernet 0/13
SW3 (config-if)# spanning-tree portfast

Warning: portfast should only be enabled on ports connected to a single host. Connecting hubs, concentrators, switches, bridges, etc... to this interface when portfast is enabled, can cause temporary bridging loops.

In addition, Cisco switches have a root guard feature to enforce the placement of root switches, preventing them from being taken over by rogue root switch.



If a root guard port receives superior BPDUs, it will transit to root inconsistent state, equivalent to listening state which stops forwarding traffic. It will recover automatically when superior BPDUs stop.

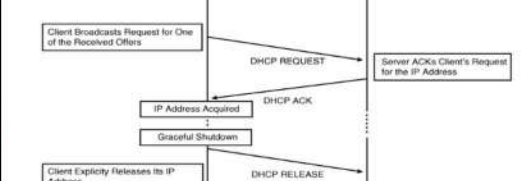
Lab Exercise 5

DHCP starvation and DHCP server spoofing attacks, which can lead to DNS server spoofing and website spoofing attacks

Defense:

Configure DHCP snooping in switches

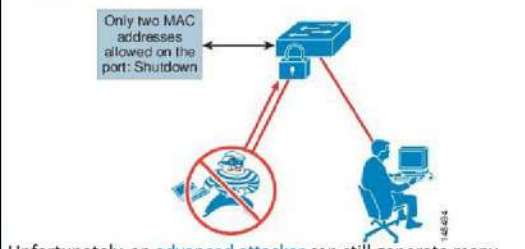
In ICT1010/ICT1013, you have learned DHCP (Dynamic Host Control Protocol) for hosts to automatically obtain IP address, default gateway, DNS, etc. in order to function.



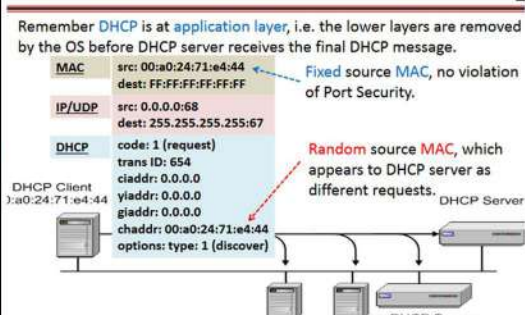
Unfortunately, DHCP has no security! An obvious attack is starvation/exhaustion/DoS attack where an attacker generates many DHCP Discover messages to seize all IP addresses.



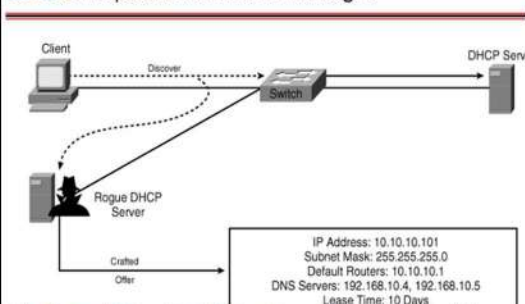
Do you think Port Security is an effective defense to counter DHCP starvation attack? Port Security limits the MAC addresses allowed on a port, which can prevent an attacker from sending many DHCP Discover messages with different MAC addresses.



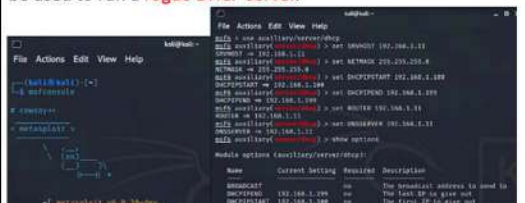
Unfortunately, an advanced attacker can still generate many DHCP Discover messages without violating Port Security as follow: (This shows why you need to know details!)



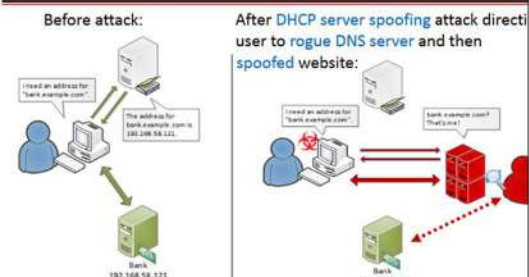
After DHCP starvation attack, an attacker may follow up with DHCP server spoofing attack which is to set up a rogue DHCP server to respond to users DHCP messages.



The Metasploit tool in Kali Linux has a DHCP module which can be used to run a rogue DHCP server.



With DHCP server spoofing attack, an attacker can now easily direct users to **rogue DNS server** to mistranslate domain name to IP address to lead them to the **spoofed website!**



An attacker may then use the information obtained to masquerade as the authorized user to login to the real website!
The **DNSChef** tool in Kali Linux can be used to run the **rogue DNS server**.

Refer to <https://www.kali.org/tools/dnschef> on dnschef usage

e.g.: To start DNS server to listen at interface 192.168.10.11 and mistranslating all DNS replies to attacker's fake IP address 192.168.10.102



To prevent **DHCP starvation** and **spoofing attacks**, a defence called **DHCP snooping** is developed which enables a switch to perform **deep-packet inspections** of higher layers.

Enable **DHCP snooping** in the switch and then specify the required VLANs as follows:

```
Switch(config)# ip dhcp snooping
Switch(config)# ip dhcp snooping vlan-id [vlan-id]
```

Configure **DHCP snooping MAC address verification** to verify layer 2 source MAC matches application layer DHCP client hardware address (chaddr), and **drop if different**.

```
Switch(config)# ip dhcp snooping verify mac-address
```

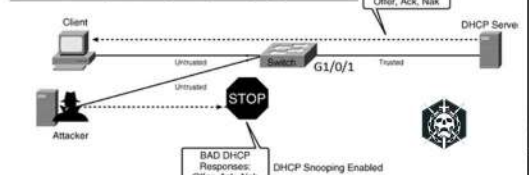
For each port, **DHCP snooping rate-limiting** (from 1 – 2048 DHCP pps) can also be configured. If **exceeded**, the port is put into **err-disabled** state.

```
Switch(config)# interface type number/number
Switch(config-if)# ip dhcp snooping limit rate rate
```

When **DHCP snooping** is enabled, **all ports** are by default **untrusted** which will **not allow** DHCP Offer and DHCP Ack messages, thus making it impossible to run rogue DHCP server.

To support DHCP servers, the **ports** connected to the **authorized DHCP servers** must be manually configured to be **trusted** as follows:

```
Switch(config)# interface type number/number
Switch(config-if)# ip dhcp snooping trust
```



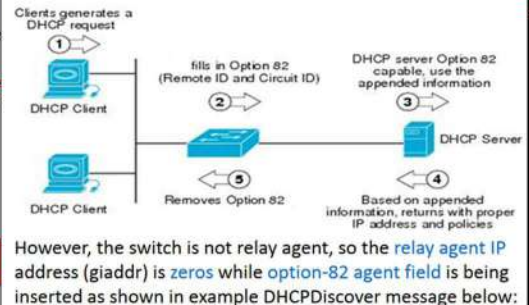
An example of configuring **DHCP snooping** on a switch and verifying the status:

```
Switch(config)# ip dhcp snooping
Switch(config)# ip dhcp snooping vlan 10
Switch(config)# interface range gigabitEthernet 1/0/35 - 36
Switch(config-if)# ip dhcp snooping limit rate 3
Switch(config-if)# interface gigabitEthernet 1/0/1
Switch(config-if)# ip dhcp snooping trust
```

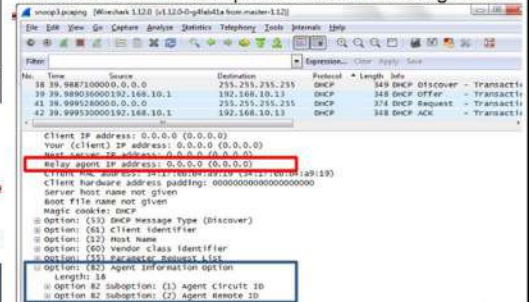
```
Switch# show ip dhcp snooping
Switch DHCP snooping is enabled
DHCP Snooping is configured on following VLANs:
10
Insertion of option 82 is enabled
Interface      Trusted      Rate limit (pps)
-----
GigabitEthernet1/0/35  no          3
GigabitEthernet1/0/36  no          3
GigabitEthernet1/0/1   yes         unlimited
```

Note that by default, a switch enabled with **DHCP snooping** will automatically insert an **option-82** (relay agent information option) field into the original DHCP messages sent by the client.

The **option-82** field contains **circuit ID** (containing VLAN and port number) and **remote ID** (containing base MAC address) of the switch which provides information about location of client that sends the DHCPDiscover message.



However, the switch is not relay agent, so the **relay agent IP address (giaddr)** is **zeros** while **option-82 agent field** is being inserted as shown in example DHCPDiscover message below:

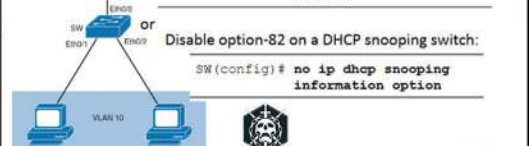


Unfortunately, a DHCP packet with **option-82** but **relay agent IP address (giaddr) zeros** will by default be **discarded** by the next receiving DHCP relay agent/server.

To resolve this problem, configure either:

```

DHCP router to trust DHCP messages:
R0(config)# ip dhcp relay information trust-all
or
R0(config-if)# ip dhcp relay information trusted
or
Disable option-82 on a DHCP snooping switch:
SW(config)# no ip dhcp snooping information option
  
```



To support **DHCP snooping**, the switch also maintains a **binding table** containing **DHCP assigned addresses** for **untrusted ports** which can be used to defend other attacks (next exercises).

The **DHCP snooping binding table** is updated automatically:

- Upon seeing **DHCPACK**, new entry is **added** into binding table.
- Upon seeing **DHCPNack**, **DHCPRelease** or **DHCPDecline**, corresponding entry is **removed**.

Macaddress	Ipdaddress	Lease(sec)	Type	VLAN	Interface
00:24:13:47:7D:B1	192.168.1.4	65859	dhcp-snooping	10	GigabitEthernet1/0/3
00:24:13:47:7D:B2	192.168.1.5	65859	dhcp-snooping	10	GigabitEthernet1/0/3
Total number of bindings: 2					

Lab Exercise 6

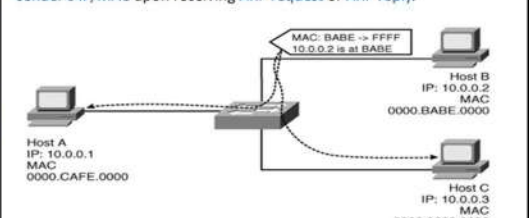
ARP poisoning attack which can lead to **MitM** (Man-in-the-Middle) attack

Defense:

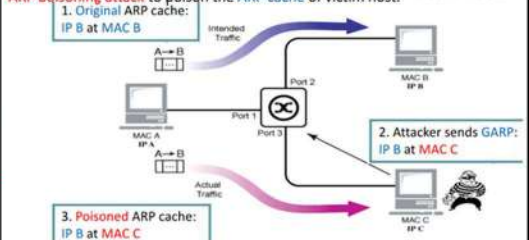
- Configure **DAI (Dynamic ARP Inspection)** in switches

In ICT1010/ICT1013, you've learned about ARP over Ethernet. In fact, there is also **gratuitous ARP (GARP)**, which is **ARP announcement** or **unsolicited ARP reply** sent without a request.

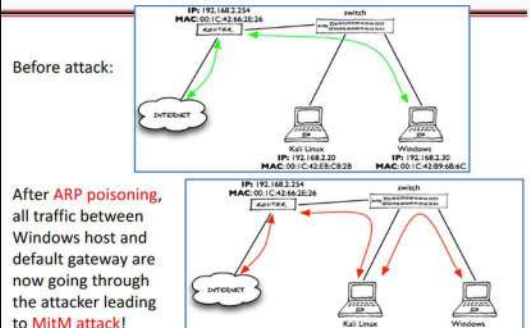
Based on RFC826, a compliant host will update its ARP cache with the **sender's IP/MAC** upon receiving **ARP request** or **ARP reply**.



Unfortunately, **gratuitous ARP** can be easily misused to perform **ARP poisoning attack** to poison the **ARP cache** of victim host.



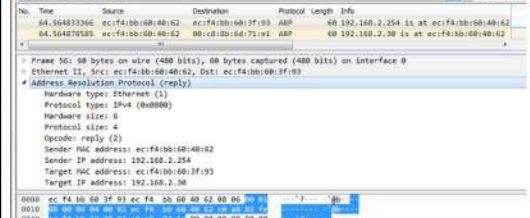
To make it worse, **ARP poisoning attack** can be extended to poison the ARP caches of victim host and default gateway simultaneously to conduct **Man-in-the-Middle (MitM) attack!**



After **ARP poisoning**, all traffic between Windows host and default gateway are now going through the attacker leading to **MitM attack!**

In Kali Linux, a tool called **Bettercap** could be used to conduct **ARP poisoning attack!**

Notice that **gratuitous ARP replies** were sent out by **Bettercap** upon initiating **ARP poisoning attack**.



After **ARP poisoning attack**, all communications of victim are redirected through attacker and could be captured.

Enabling **sniffing** in Bettercap:

```

bettercap -i eth0 --sniff --local true
bettercap -i eth0 --sniff --local true --sniff --local true
bettercap -i eth0 --sniff --local true --sniff --local true
bettercap -i eth0 --sniff --local true --sniff --local true
bettercap -i eth0 --sniff --local true --sniff --local true
  
```

Username and **password** could be **captured** if not encrypted!



You may also try **driftnet** in Kali to sniff and display the images that the victim is seeing!
Even if visiting https website, an attacker may try **SSL stripping** or **HSTS bypass attacks!**

HSTS (HTTP Strict Transport Security) (RFC 6797) is a security mechanism for web sites to declare to browsers to access via secure connections on

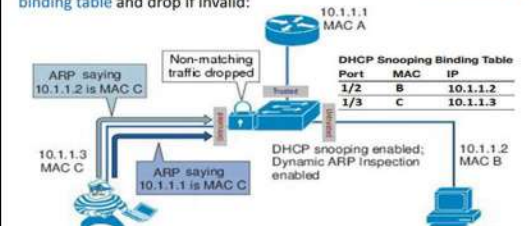


Instead of <https://www.google.com> notice that victim is redirected to www.google.com.



To prevent **ARP poisoning attack**, a switch enabled with **DHCP snooping** can be further configured with **DAI (Dynamic ARP Inspection)** to drop invalid ARP traffic on **untrusted ports**.

After enabling **DAI**, all ports are **untrusted** by default similar as **DHCP snooping**. The **MAC** and **IP addresses** in the ARP packets received on these **untrusted ports** will then be validated against the **DHCP snooping binding table** and drop if invalid:



An example of configuring **DAI** together with **DHCP snooping** on a switch and verifying the status:

```

SW(config)# ip dhcp snooping
SW(config)# ip dhcp snooping vlan 10
SW(config-if)# ip dhcp snooping limit rate 10
SW(config-if)# interface fa1/1
SW(config-if)# ip dhcp snooping trust
SW(config-if)# ip arp inspection trust
  
```

Each port can also be configured with **ARP rate-limit** (default is 15 ARP pps). If exceeded, the port will transit to **err-disabled** state.

```
Switch(config-if)# ip arp inspection limit (rate pps [burst interval seconds] | none)
```


An example of configuring **extended named ACL**:

- ```
Router(config)# ip access-list extended barney
Router(config-ext-nacl)# permit tcp host 10.1.1.2 eq www any
Router(config-ext-nacl)# deny udp host 10.1.1.1 10.1.2.0 0.0.0.255
Router(config-ext-nacl)# deny ip 10.1.3.0 0.0.0.255 10.1.2.0 0.0.0.255
Router(config-ext-nacl)# deny ip 10.1.2.0 0.0.0.255 10.2.3.0 0.0.0.255
Router(config-ext-nacl)# permit ip any any
Router(config-ext-nacl)# interface serial1
Router(config-if)# ip access-group barney out
```
- In newer versions of Cisco IOS, **numbered ACLs** are also able to be configured in the **same way as named ACLs**.  
 Just use **number** instead of name to configure **numbered ACL** using the same **style** as named ACL:

```

[Step 1: The Standard Numbered IP ACL is configured.]
R1# configure terminal
Enter configuration commands, one per line. End with Ctrl-Z.
R1(config)# ip access-list standard 24
R1(config-std-nacl)# permit 10.1.1.0 0.0.0.255
R1(config-std-nacl)# permit 10.1.2.0 0.0.0.255

```



Note that sequence numbers are automatically inserted by IOS:

```
R1(config-std-nacl)# permit 10.1.3.0 0.0.0.255

Step 2: Displaying the ACL's contents, without leaving configuration mode.

R1(config-std-nacl)# do show ip access-list 24

Standard IP access list 24

10 permit 10.1.1.0, wildcard bits 0.0.0.255
20 permit 10.1.2.0, wildcard bits 0.0.0.255
30 permit 10.1.3.0, wildcard bits 0.0.0.255
```

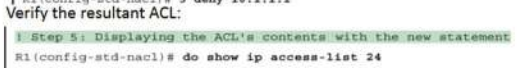
With sequence numbers being inserted automatically, both **named** and **sequence-numbered** ACLs can be **edited** easily as follows:  
To **delete** ACL statement, use **no** sequence-number command:

Step 3: Still in ACL 24 configuration mode, the line with sequence number 20 is deleted.

```
R1(config-std-nacl)# no 20
```

To insert new ACL statement, configure it starting with a sequence

```
1 Step 4: Inserting a new first line in the ACL.
2 [config-std-pac]# 5 deny 10.1.1.1
```



```
Standard IP access list 24
5 deny 10.1.1.1
10 permit 10.1.1.0, wildcard bits 0.0.0.255
10 permit 10.1.3.0, wildcard bits 0.0.0.255
```

In addition, the automatic numbering of the **ACL statements** can be **resequenced** as follows, e.g. if you need to insert more ACL statements in between:

Step 6: resequence the ACL's contents

starting sequence number      increment step

```
! Step 7: Displaying the ACL's contents one last time
R1(config)# do show ip access-list 24
```

```
Standard IP access list 24
 10 deny 10.1.1.1
 30 permit 10.1.1.0, wildcard bits 0.0.0.255
```

50 permit 10.1.3.0, wildcard bits 0.0.0.255

(4.4) Time-based ACLs are basically ACLs enhanced with time-range which will only become operational during certain time of

For example, a company may wish to only allow access to its internal web server during office hours. If without time-based ACL, it will be

Only permit connections to the internal web server Mon-Fri from 9am-5pm.

Diagram illustrating a Remote User attempting to reach the internal Corporate Web through a router (R1). The router has an S1/0 interface connected to the user and an S0/0 interface connected to the Corporate Web. A cloud icon represents the Internet.

The diagram shows a blue switch icon connected by a line to a blue server icon labeled "Server".

Time-range is referencing the router's internal clock, which for accuracy can be synchronized using NTP (to be covered in later labs).  
An example of configuring a time-based ACL:

Only permit connections to the internal web server Mon-Fri from 8:00 am-5:00 pm.

(Config Case-Large) **periodic** 1  
 Friday Friday  
 Monday Monday  
 Saturday Saturday  
 Sunday Sunday

The diagram shows a router labeled R1. A lightning bolt symbol represents an external connection. A line labeled S1/0 connects the router to a box labeled "Remote User Attempting to Reach the Internal Corporate Web".

Thursday Thursday  
Tuesday Tuesday  
Wednesday Wednesday  
daily Every day of the week  
weekdays Monday thru Friday



Server

```

weekend Saturday and Sunday
Internal Web Server
192.168.1.10
R1(config-time-range)# periodic weekdays 8:00 to 17:00
R1(config-time-range)# exit
R1(config)# access-list 100 permit tcp any host 192.168.1.10 eq 80 time-range

```

```
... OUTPUT OMITTED FOR OTHER PERMIT ACL STATEMENTS NOT RELEVANT TO THIS EXAMPLE ...

R1(config)# interface serial 1/0
```

```

R1(config)# interface GigabitEthernet 1/0/24
R1(config-if)# ip access-group 100 in
R1(config-if)# end

```

(4.5) To be more readable, a new way of writing ACL called **named ACL** has been introduced in Cisco routers.

Object group-based ACL has been introduced in Cisco routers and ASA firewalls.

- IP ranges
- subnet
- etc
- Service
  - protocol



- port number
- etc
- e.g. tcp www

Note: User, ICMP-type and Security object groups are less commonly

An example of configuring network object group.

```
router(config)# object-group network Inside_Eng
router(config-network-group)# 10.0.0.0 /24
router(config-network-group)# exit
router(config)# object-group network Inside_Mktg
router(config-network-group)# host 10.0.1.1
router(config-network-group)# host 10.0.1.2
router(config-network-group)# exit
router(config)# object-group network Inside_Networks
router(config-network-group)# group-object Inside_Eng
router(config-network-group)# group-object Inside_Mktg
```

Notes:

- Object group may be **nested** using the **group-object** command
- Configuration commands on ASA firewalls are slightly different

An example of configuring **service object group**.

Host Services

- HTTP
- HTTPS
- FTP

```
router(config)# object-group service Host_Services
router(config-service-group)# tcp www
router(config-service-group)# tcp 443
router(config-service-group)# tcp ftp
```

Note: Configuration commands on ASA firewalls are slightly different, use 7 for help.

Finally, configure **object group-based ACL** based on the configured **network** and **service object groups** as follows:

```
router(config)# int G0/0
router(config-if)# ip
access-group 100 out
```

Lab 4b

In contrast to internal attacks, the first point of entry for **external attackers** are typically the **routers** and **firewalls**.

Untrusted Network (premises screening) Router Firewall Corporate (trusted) Network

Internet DMZ Web Server Mail Server

To prevent **external attackers**, **ACLs** (access control lists) are commonly implemented at the **perimeter/edge router** and **firewall** which connect the enterprise network to the Internet.

Lab Exercise 1

1.1 Network reconnaissance - ping sweeping attack

1.2 Mitigation:

- Apply **ACL** to filter network traffic

A common external attack is **network reconnaissance** using **ping sweeping** and **port scanning** to discover targets which you'll learn more in ICT2204/ICT2212 Ethical Hacking.

Technically, **ping sweeping** and **port scanning** are based on sending **ARP**, **ICMP**, **TCP** or **UDP** requests and receiving/not receiving the replies.

Example of **ICMP scan**:

Example of **ARP scan**:

Example of **UDP scan**:

A useful tool for conducting **ping sweeping** and **port scanning** is **nmap** which you will try in the lab.

**Nmap**

Basic Syntax

Target Specification

Probe Options

Scan Types

Output Formats

Script Options

SANS

Unfortunately, this can result in **port scanning**, but **ping sweeping** may be blocked using suitable **ACL**; e.g. **ingress filtering of ICMP scan** as follows:

Diagram showing traffic flow from Internet to Perimeter Router to Hosts.

```
R1(config)# ip access-list extended nmapICMPscan-in
R1(config-ext-nacl)# deny icmp any any echo
R1(config-ext-nacl)# deny icmp any any timestamp-request
R1(config-ext-nacl)# deny icmp any any mask-request
R1(config-ext-nacl)# permit ip any any
R1(config)# interface E1
R1(config-if)# ip access-group nmapICMPscan-in in
R1(config-if)# no ip unreachable
```

Recall ICT1010/ICT1013 on ICMP error-reporting messages?

Note that there are different **type 3 destination unreachable** error messages for different error conditions (codes) as shown:

| Type                      | Code | Description                               |
|---------------------------|------|-------------------------------------------|
| 3 Destination Unreachable | 0    | Destination network unreachable           |
|                           | 1    | Destination host unreachable              |
|                           | 2    | Destination protocol unreachable          |
|                           | 3    | Destination port unreachable              |
|                           | 4    | Fragmentation required, and DF flag set   |
|                           | 5    | Source route failed                       |
|                           | 6    | Destination network unknown               |
|                           | 7    | Destination host unknown                  |
|                           | 8    | Source host isolated                      |
|                           | 9    | Network administratively prohibited       |
|                           | 10   | Host administratively prohibited          |
|                           | 11   | Network unreachable for ToS               |
|                           | 12   | Host unreachable for ToS                  |
|                           | 13   | Communication administratively prohibited |
|                           | 14   | Host precedence violation                 |
|                           | 15   | Precedence cutoff in effect               |

Host A attempts to send an IP packet to Host B. Host B is **unreachable**, so the router responds with a **Type 3 ICMP message**.

**No ip unreachable**s will prevent a router configured with ACL to drop packets from sending **ICMP type 3 destination unreachable** which will leak information to attackers, e.g. shown below:

Wireshark capture showing ICMP Echo (ping) request and response.

In addition, you may want to implement **egress filtering of ICMP** to prevent someone inside your network from conducting **ping sweeping** over Internet, be it intentionally or unknowingly.

```
R1(config)# ip access-list extended ICMPscan-out
R1(config-ext-nacl)# permit icmp host 200.1.1.5 any echo
R1(config-ext-nacl)# permit icmp 200.1.1.0 0.0.0.255 any
R1(config-ext-nacl)# replace-with-actual-ICMP-error-reporting-message-see-below
R1(config-ext-nacl)# deny icmp any any
R1(config-ext-nacl)# permit ip any any
R1(config)# interface E1
R1(config-if)# ip access-group ICMPscan-out out
```

1 Allow administrator to ping which is useful for testing connectivity.

2 Trade-off allowing **ICMP error-reporting messages** vs **attacks**; e.g.

- unreachable**: inform destination unreachable but exploited by **UDP scan**
- time-exceeded**: exploited by attacker to **trace route** to target

Lab Exercise 2

2.1 Denial-of-Service (DOS) / Distributed DoS (DDoS) attack - reflection and amplification attack such as **Smurf attack** using **spoofed IP address** of target

2.2 Mitigation for reflection and amplification attack:

- Apply **ACL** as recommended by **BCP 38/RFC 2827** to filter **spoofed IP addresses**
- Apply **anti-spoofing ACL** as added security
- Additional mitigation for **Smurf attack**:
  - Disable hosts from replying to **broadcast ping**
  - Disable routers from supporting **directed broadcast**

Typically, **DoS/DDoS** aims to overload the **network bandwidth** or **resources** of the target to make it unavailable.

Diagram showing Attacker sending traffic to Victim's Infrastructure.

**DDoS** attack is basically **DoS** attacks that are launched using an army of compromised hosts called **zombies/bots** distributed in the Internet. Broadly, **DoS/DDoS attacks** may be classified into 2 categories as follows:

| DoS/DDoS categories              | Description                                                                                                                                                                             | Typical attack packet rate | Examples                                           |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|----------------------------------------------------|
| Overwhelming Quantity of Traffic | Threat actor sends an enormous quantity of data at a rate that the network, host, or application cannot handle, possibly causing slow response times or even crash a device or service. | High                       | ICMP flood, UDP flood, smurf attack, TCP SYN, etc. |
| Maliciously Crafted Packets      | Threat actor sends maliciously crafted packets that the host or application is not expected to handle, possibly causing it to run very slowly or even crash.                            | Low                        | Slowloris, ping of death, etc.                     |

**Reflection and amplification attack** is a type of **DDoS** attack by sending packets to intermediary servers using **spoofed source IP** of target so that replies are **reflected** and **amplified** to target.

Diagram showing Attacker sending traffic to Servers, which then reflect and amplify traffic to Target IP Router and Private IP Network.

An example of reflection and amplification is **Smurf attack** which appeared in the late 1990s. Today, reflection and amplification attacks using **DNS** or **NTP** servers are still possible. In **Smurf attack**, a single **broadcast ping** is sent to a **directed broadcast address** by attacker using **spoofed IP source address** belonging to target. would result in multiple ping replies (each from an intermediary in the directed broadcast subnet) reflected to the target.

Diagram showing Attacker sending ICMP ECHO SRC=200.1.1.1 DST=171.125.255 to Intermediaries, which then reflect traffic to Target.

As it can be illegal to try **reflection and amplification attack** using **DNS** or **NTP** servers in the Internet, you will try **Smurf attack** in the lab using **hping3**:

Refer <https://www.kali.org/tools/hping3> on the use of hping3:

```
root@kali:~# hping3 -l -flood -a target_ip directed_broadcast_address
```

Today, **Smurf attack** has been largely prevented by default through:

- Disable routers from supporting **directed broadcast**;
- Disable hosts from replying to **broadcast ping**.

To prevent reflection and amplification attacks using **spoofed source IP addresses** of targets, ISPs today are recommended to implement Best Current Practice **BCP 38/RFC 2827**. BCP 38/RFC 2827 recommends ISPs to implement **network ingress filtering** to prevent their networks from permitting spoofed IP addresses and supporting **DoS/DDoS** attacks.

BCP 38 Filter = Allow Only Source Addresses from the Customer's 96.0.X.X/24

Diagram showing Internet, ISP Core, and customers (96.0.20.0/24, 96.0.21.0/24, 96.0.19.0/24, 96.0.18.0/24).

access-list 121 permit ip 96.0.18.0 0.0.0.255 any  
access-list 121 deny ip any any log  
interface serial 1/1/3  
ip access-group 121 in

BCP 38 Filter Applied on Router

Similarly, organizations should also implement **BCP 38/RFC 2827** at their edge routers to prevent their hosts, possibly zombies, from using **spoofed source IP addresses** to launch attacks.

Enterprise: 96.0.18.0/24

Block Leaving Source # Own Network  
access-list 102 permit ip 96.0.18.0 0.0.0.255 any  
access-list 102 deny ip any any

Block Sources That Do Not Belong to Subnet  
access-list 102 permit ip 192.168.0.0 0.0.0.255 any  
access-list 102 deny ip any any

In addition, organizations should implement **anti-spoofing ACL** at their **edge routers** to prevent being victim to attacks using spoofed IP addresses.

Diagram showing ISP, Enterprise (96.0.18.0/24), and LANs (192.168.1/24, 192.168.2/24, 192.168.3/24).

Block Entering Source # Own Network  
access-list 101 deny ip 96.0.18.0 0.0.0.255 any  
access-list 101 permit ip any any

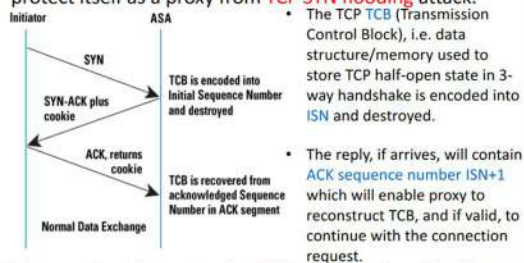
**Anti-spoofing ACL** is used to deny inbound IP packets that are obviously spoofed; e.g. source addresses that match your allocated public IP addresses, or special addresses in **RFC 3330** and **RFC 1918** that should not be used.

Typically, **anti-spoofing ACL** is implemented as a part of overall **infrastructure protection ACL** at the edge router for **ingress filtering** of malicious traffic from entering the network.





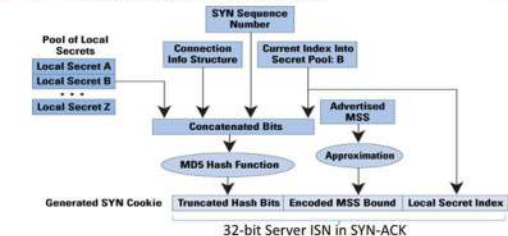
In addition, ASA implements **TCP SYN cookie** mechanism to protect itself as a proxy from **TCP SYN flooding** attack.



- The TCP TCB (Transmission Control Block), i.e. data structure/memory used to store TCP half-open state in 3-way handshake is encoded into ISN and destroyed.
- The reply, if arrives, will contain **ACK sequence number ISN+1** which will enable proxy to reconstruct TCB, and if valid, to continue with the connection request.

The procedure for generating **SYN cookie** varies with different implementations, an example as follows:

- Connection Info Structure: IP addresses and TCP ports
- SYN Sequence Number: 32-bit Client ISN



- To validate reply, decrease ACK number - 1 to recover **SYN cookie**, and **local secret index**, then apply the same procedure. If valid, allow TCP connection to internal server.

To verify and monitor attacks intercepted by **TCP intercept**, you may configure **threat detection statistics** as follows:

- Enabling **threat detection statistics** for TCP intercept:
- ```

ciscoasa(config)# threat-detection statistics tcp-intercept
rate-interval 3 burst-rate 60 average-rate 30

```
- rate-interval** sets the size of monitoring window in **mins** during which ASA samples the attacks 60 times
 - burst-rate** and **average-rate** are in attacks/events per sec (**eps**), when exceeded will generate syslog message
 - Monitoring **TCP intercept statistics**:

```

ciscoasa(config)# show threat-detection statistics top tcp-intercept
Top 10 protected servers under attack (sorted by average rate)
Monitoring window size: 3 mins Sampling interval: 3 secs
<Rank> <Server IP:Port> <Interface> <Avg Rate> <Cur Rate> <Total> <Source IP (Last Attack Time)>
1 10.1.1.5:80 inside 1249 9503 2249245 10.0.0.3 (0 secs ago)

```

Lab Exercise 2

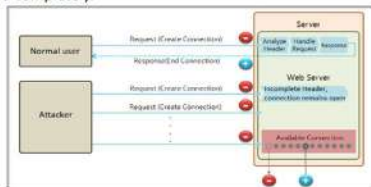
2.1 Slowloris attack

2.2 Defense:

- Configure **MPF service policy** on ASA

Slowloris attack is an example of **DoS** attack on web servers using low-rate but maliciously crafted packets to exploit design weakness of HTTP protocol.

By design, web server will only process HTTP request when header is received completely.



By creating **many concurrent connections** to the web server, and for each connection, intentionally **sending HTTP header slowly** in many small fragments, **Slowloris** is thus able to consume all connections and make the server unavailable.

To try **Slowloris** attack in the lab, you may use the **slowhttptest** in Kali Linux.

```
root@kali:~# slowhttptest -c number_connections -u target_url
```

Refer <https://www.kali.org/tools/slowhttptest/> on the use of slowhttptest.

An example of **Slowloris** attack on XAMPP Apache web server with default setting.

Notice that a single client making 150 connections is sufficient to make Apache web server unavailable!

Similar as defending TCP SYN attack, **MPF inspection security policy** may be configured to defend Slowloris. Try it yourself!

```

root@kali:~# slowhttptest -c 150 -u http://192.168.1.100
Wed Oct 2 13:26:15 2019:
slowhttptest version 1.8
- https://code.google.com/p/slowhttptest/ -
Test type: SLOW HEADERS
URL: http://192.168.1.100/
Server: Apache/2.4.6 (Ubuntu)
Request Length: 1024 bytes
Interval: 10 seconds
Connections: 150
Status: 0
Error: 0
Closed: 0
Service available: no

```